

LAPORAN TUGAS PROYEK BIG DATA

IF4044 TEKNOLOGI BIG DATA



Disusun oleh:

Kelompok 2

Irfan Sidiq Permana	13522007
Ellijah Darrellshane S.	13522097
Mayla Yaffa Ludmilla	13523050

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2026

1. Pendahuluan

1.1. Latar Belakang

Pada proyek ini dilakukan pembangunan arsitektur big data menggunakan konsep Data Lakehouse, Data Mart, dan Data Orchestration. Dataset yang digunakan adalah dataset TPC-H yang digenerate menggunakan DBGEN dalam format CSV. Pipeline data mencakup proses generate data, load data ke Data Lakehouse, transformasi data menggunakan dbt, hingga pembentukan Data Mart untuk kebutuhan analisis.

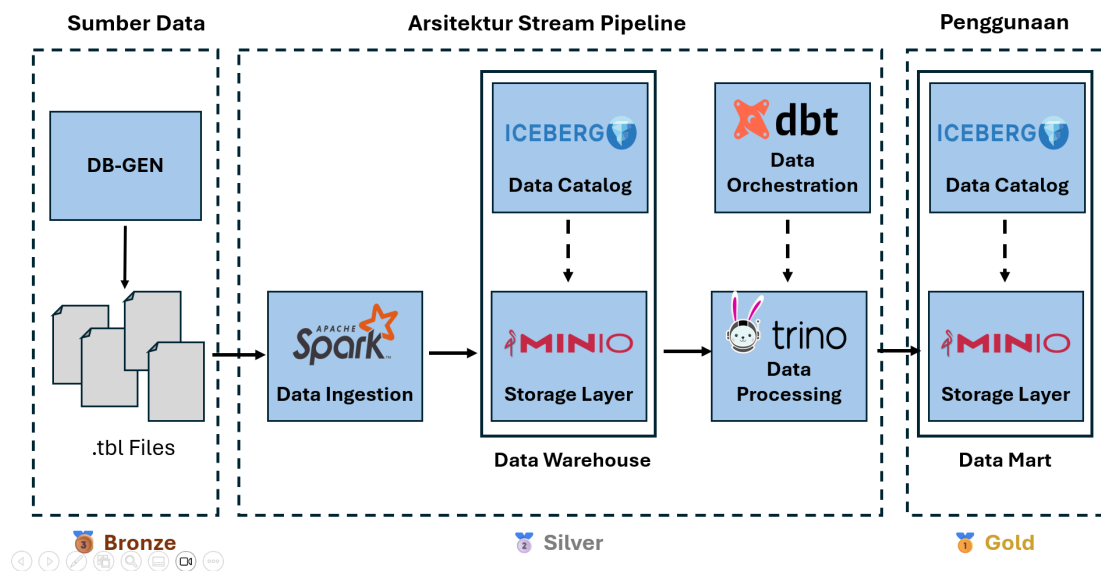
1.2. Tujuan

Tujuan dari proyek ini adalah:

- Mengimplementasikan arsitektur Data Lakehouse end-to-end
- Memproses dataset TPC-H menggunakan tools big data
- Menggunakan dbt untuk transformasi dan orkestrasi data
- Mengevaluasi efisiensi format Parquet dibanding CSV
- Menerapkan praktik data engineering yang maintainable dan reusable

2. Arsitektur dan Teknologi Sistem

2.1. Gambaran Umum Arsitektur



Gambar 2.1 Arsitektur Sistem Proyek

Secara umum, arsitektur yang dibangun pada proyek ini terdiri dari beberapa tahapan utama yang saling terhubung dalam sebuah pipeline data lakehouse. Pipeline dimulai dari proses generate data hingga transformasi data untuk

kebutuhan analytics dan data mart. Adapun alur sistem yang diimplementasikan adalah sebagai berikut:

1. Generate data TPC-H menggunakan DBGEN
2. File CSV disimpan pada MinIO sebagai object storage
3. Spark melakukan ingest data ke Apache Iceberg
4. Data disimpan dalam format Parquet pada lakehouse
5. Trino digunakan untuk menjalankan query analytics
6. dbt digunakan untuk membangun data mart dan transformasi data

2.2. Teknologi yang Digunakan

- **MinIO**

MinIO digunakan sebagai object storage utama dalam arsitektur lakehouse yang dibangun. MinIO dipilih karena mendukung API yang kompatibel dengan Amazon S3 sehingga dapat dengan mudah diintegrasikan dengan berbagai tools big data seperti Apache Iceberg, Spark, maupun Trino. Selain itu, MinIO relatif ringan, mudah dikonfigurasi, serta dapat dijalankan secara lokal menggunakan Docker, sehingga cocok untuk kebutuhan pengembangan dan eksperimen proyek skala kecil hingga menengah tanpa memerlukan infrastruktur cloud yang kompleks.

Di proyek ini, MinIO berfungsi sebagai tempat penyimpanan dataset TPC-H baik dalam bentuk CSV maupun file Parquet hasil ingest ke lakehouse. Dengan pendekatan object storage, data dapat diakses oleh berbagai komponen pipeline secara terpusat tanpa perlu dipindahkan antar service sehingga alur pipeline menjadi lebih efisien dan scalable.

- **Apache Iceberg**

Apache Iceberg digunakan sebagai format table pada arsitektur data lakehouse. Iceberg dipilih karena memiliki fitur modern seperti schema evolution, partition management, hidden partitioning, serta metadata management yang membuat pengelolaan data dalam jumlah besar menjadi lebih efisien dibanding penyimpanan file biasa. Selain itu, Iceberg mendukung ACID transaction sehingga konsistensi data tetap terjaga saat proses baca dan tulis dilakukan secara bersamaan.

Di proyek ini, Iceberg digunakan untuk menyimpan data hasil ingest dari Spark ke dalam format Parquet. Penggunaan Iceberg membantu proses query analytics menjadi lebih cepat dan terstruktur karena metadata tabel dikelola

secara otomatis. Selain itu, Iceberg juga mempermudah integrasi dengan query engine seperti Trino dan proses transformasi menggunakan dbt.

- **Apache Spark**

Apache Spark digunakan sebagai data processing engine pada pipeline yang dibangun. Spark dipilih karena mampu memproses data dalam jumlah besar secara distributed serta memiliki integrasi yang baik dengan format lakehouse seperti Apache Iceberg.

Di proyek ini, Spark digunakan untuk membaca dataset CSV dari MinIO kemudian melakukan ingest data ke dalam Iceberg table dengan format Parquet. Spark juga membantu proses transformasi awal data sebelum data digunakan lebih lanjut untuk analytics dan data mart.

- **Trino**

Trino digunakan sebagai SQL query engine untuk menjalankan query analytics pada data lakehouse. Trino dipilih karena mampu melakukan query secara cepat langsung pada data yang tersimpan di object storage tanpa perlu memindahkan data ke database tradisional.

Di proyek ini, Trino digunakan untuk menjalankan query analytics seperti aggregation dan group-by pada tabel Iceberg. Penggunaan Trino mempermudah eksplorasi data karena query dapat dilakukan menggunakan syntax SQL yang familiar dengan tetap mendapatkan performa yang baik pada dataset berukuran besar.

- **dbt (Data Build Tool)**

dbt digunakan sebagai tools transformasi dan orchestration data pada pipeline yang dibangun. dbt dipilih karena memungkinkan proses transformasi dilakukan menggunakan SQL dengan struktur project yang modular dan mudah dipelajari.

Di proyek ini, dbt digunakan untuk membangun data mart dari tabel-tabel Iceberg yang sudah tersedia. Selain transformasi data, dbt juga membantu meningkatkan maintainability pipeline melalui penggunaan model, macros, schema.yml, dan data testing seperti unique serta not_null. Dengan fitur tersebut, pipeline data menjadi lebih readable, reusable, dan mudah dikembangkan di kemudian hari.

2.3. Spesifikasi Mesin yang Digunakan

Pada tugas besar ini, kami semua melakukan *setup* pada laptop kami masing-masing agar tiap anggota memiliki kesempatan mencoba. Namun, khusus untuk melakukan uji ukuran file dan uji *query performance*, hasil yang disertakan di laporan ini adalah hasil dari *setup* salah satu anggota saja dengan spesifikasi laptop sebagai berikut:

Komponen	Spesifikasi
CPU	12th Gen Intel(R) Core(TM) i5-12450HX (2.40 GHz)
RAM	12,0 GB (11,7 GB usable)
Storage	512GB M.2 2242 PCIe 4.0x4 NVMe SSD
OS	Windows 11 Home Edition
Versi WSL	2.3.26.0
Alokasi Docker	8 GB RAM

3. Langkah Data Processing

3.1. Generate Dataset

```
./dbgen -s 10
```

Command ini menjalankan DBGEN, sebuah data generator yang disediakan oleh TPC (Transaction Processing Performance Council) untuk membuat dataset sintetis TPC-H. Flag `-s 10` menentukan scale factor atau ukuran data yang dihasilkan. Di tugas besar ini, digunakan scale factor sebesar 10 yang berarti data yang di-generate akan berukuran sekitar 10GB. DBGEN bekerja dengan menggunakan formula matematika dan seed tertentu untuk menghasilkan data yang terdistribusi secara realistis, mencakup 8 tabel relasional seperti orders, lineitem, customer, part, dan lainnya. Output dari command ini akan berupa file `.tbl` yang formatnya mirip CSV, tetapi dengan delimiter `|`.

```
eline/Kode/tpch-dbggen$ ./dbgen -s 10
TPC-H Population Generator (Version 2.14.0)
Copyright Transaction Processing Performance Council 1994 - 2010
```

Gambar 3.1 Eksekusi command untuk generate data

```
henryofskalitz@DarrellShane:/mnt/c/Users/edarr/Downloads/BigDataPipeline/data/raw_tbl$ ls -lh *.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 234M May 11 11:12 customer.tbl
-r-xr-xr-x 1 henryofskalitz henryofskalitz 7.3G May 11 11:12 lineitem.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 2.2K May 11 11:12 nation.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 1.7G May 11 11:12 orders.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 233M May 11 11:12 part.tbl
-r-xr-xr-x 1 henryofskalitz henryofskalitz 1.2G May 11 11:12 partsupp.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 389 May 11 11:12 region.tbl
-rwxrwxrwx 1 henryofskalitz henryofskalitz 14M May 11 11:12 supplier.tbl
henryofskalitz@DarrellShane:/mnt/c/Users/edarr/Downloads/BigDataPipeline/data/raw_tbl$
```

Gambar 3.2 Hasil command untuk generate data

3.2. Inisialisasi Infrastruktur (Docker)

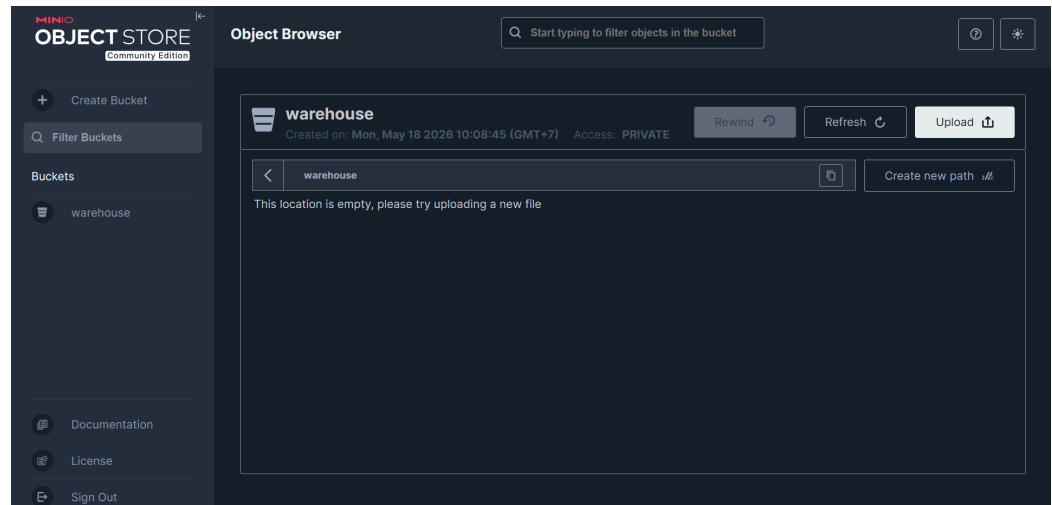
```
docker-compose up -d
```

Command ini membaca file docker-compose.yml yang berisi definisi seluruh service yang dibutuhkan dalam pipeline ini yaitu MinIO, Spark, Trino, Iceberg dan lainnya. Docker Compose kemudian membuat network virtual yang menghubungkan semua container tersebut sehingga bisa saling berkomunikasi satu sama lain lewat nama service-nya (bukan IP). Flag -d (detached) membuat semua container berjalan di background sehingga terminal tetap bisa digunakan. Setiap container di-spin up dari Docker image yang sudah dikonfigurasi sebelumnya di docker-compose.yml, termasuk environment variable, volume mount, dan port mapping-nya.

```
PS C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode> docker-compose up -d
[+] Running 6/6
✓ Network kode_lakehouse_net Created
✓ Container minio Started
✓ Container spark-iceberg Started
✓ Container rest Started
✓ Container minio-init Started
✓ Container trino Started
```

Gambar 3.3 Eksekusi command Docker

Setelah menjalankan container Docker, MinIO Web UI akan dapat diakses melalui browser di <http://localhost:9001>. Web UI ini memungkinkan kita untuk memverifikasi isi bucket secara visual tanpa perlu menggunakan CLI.



Gambar 3.4 Tampilan MiniIO Web UI

3.3. Load Data ke Lakehouse

```
docker exec -it spark-iceberg spark-submit
/home/iceberg/ingestion/loader.py
```

Command ini digunakan untuk mengeksekusi perintah di dalam container yang sudah berjalan, dalam hal ini container spark-iceberg. spark-submit adalah CLI tool milik Apache Spark untuk men-submit aplikasi Spark yaitu berupa script Python loader.py. Ketika dijalankan, Spark membaca file .tbl dari direktori yang sudah dimasukkan ke container, melakukan parsing schema sesuai definisi masing-masing tabel TPC-H, lalu mengonversinya dan menuliskan hasilnya ke MinIO dalam format Parquet. Apache Iceberg berperan sebagai lapisan metadata di atas file Parquet tersebut yang menyimpan informasi schema, partisi, dan snapshot history sehingga data yang tersimpan di MinIO bisa diperlakukan layaknya tabel terstruktur oleh query engine seperti Trino.

```
(venv) PS C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode> docker exec -it spark-iceberg spark-submit /home/iceberg/ingestion/loader.py

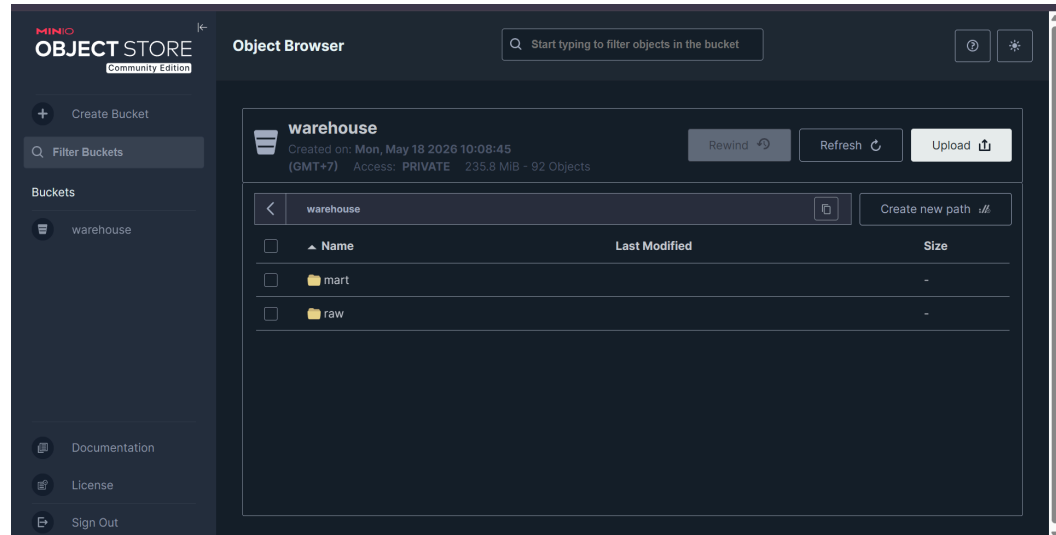
=====
=== STARTING INGESTION PIPELINE (PATCHED DEMO S3) ===
=====
```

Gambar 3.5 Command load data dimulai

```
[PROSES INGESTION SELESAT SEPENUHNYA]
26/05/20 15:45:58 INFO ShutdownHookManager: Shutdown hook called
26/05/20 15:45:58 INFO ShutdownHookManager: Deleting directory /tmp/spark-8710032b-01d7-4543-a149-3a70bb8dd345
26/05/20 15:45:58 INFO ShutdownHookManager: Deleting directory /tmp/spark-cd9c5042-3d2d-4c1b-8a1a-7bedabc2aeac
26/05/20 15:45:58 INFO ShutdownHookManager: Deleting directory /tmp/spark-8710032b-01d7-4543-a149-3a70bb8dd345/pyspark-9089521f-9145-4775-a6c3-289512965800
```

Gambar 3.6 Command load data tereksekusi

Setelah ingestion selesai dapat diverifikasi di MinIO Web UI bahwa bucket warehouse sudah berisi file-file Parquet yang terorganisir per tabel.



Gambar 3.7 Tampilan MiniIO Web UI

3.4. Transformasi dengan dbt

```
dbt clean
```

Command ini menghapus folder target/ dan dbt_packages/ yang merupakan hasil kompilasi dari run sebelumnya atau memastikan bahwa tidak ada artifact lama yang bisa menyebabkan inkonsistensi saat menjalankan ulang pipeline.

```
o (venv) PS C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project> dbt clean
16:12:09 Running with dbt=1.11.10
16:12:09 Checking C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project\target/*
16:12:09 Cleaned C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project\target/*
16:12:09 Checking C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project\dbt_packages/*
16:12:09 Cleaned C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project\dbt_packages/*
16:12:09 Finished cleaning all paths.
```

Gambar 3.8 Eksekusi command dbt clean

```
dbt run
```

Command ini membaca seluruh file model SQL yang ada di folder models/. Sebelum dieksekusi, dbt terlebih dahulu mengkompilasi file-file tersebut. Setelah itu, dbt mengirimkan query SQL hasil kompilasi tadi ke Trino sebagai query engine-nya. Trino kemudian membaca data dari tabel-tabel Iceberg yang tersimpan di MinIO, memproses query-nya, lalu menuliskan hasilnya sebagai

tabel baru di Iceberg. Tabel ini adalah tabel yang membentuk Data Mart. Selama proses ini, dbt juga membaca file model.yml untuk menerapkan metadata seperti deskripsi tabel dan kolom ke tabel yang dibuat, sehingga Data Mart yang dihasilkan sudah terdokumentasi secara otomatis.

```
(venv) PS C:\Users\Mayla\Documents\Kuliah\Semester-6\Teknologi Big Data\BigDataPipeline\Kode\dbt_project> dbt run
14:04:12 Running with dbt=1.11.10
INFO:trino.auth:keyring module not found. OAuth2 token will not be stored in keyring.
INFO:trino.auth:keyring module not found. OAuth2 token will not be stored in keyring.
14:04:12 Registered adapter: trino=1.10.1
14:04:12 Unable to do partial parsing because saved manifest not found. Starting full parse.
14:04:14 Found 1 model, 3 sources, 487 macros
14:04:14 Concurrency: 4 threads (target='dev')
14:04:14
14:04:17 [WARNING]: SSL certificate validation is disabled by default. It is legacy behavior which will be changed in future releases. It is strongly advised to enable 'require_certificate_validation' flag or explicitly set 'cert' configuration to 'True' for security reasons. You may receive an error after that if your SSL setup is incorrect.
You may opt into the new behavior sooner by setting 'flags.require_certificate_validation' to 'True' in 'dbt_project.yml'.
Visit https://docs.getdbt.com/reference/global-configs/behavior-changes for more information.
14:04:20 1 of 1 START sql table model mart.mart_customer_revenue ..... [RUN]
14:04:32 1 of 1 OK created sql table model mart.mart_customer_revenue ..... [CREATE TABLE (99_996 rows) in 11.84s]
14:04:32
14:04:32 Finished running 1 table model in 0 hours 0 minutes and 18.67 seconds (18.67s).
14:04:32
14:04:32 Completed successfully
14:04:32
14:04:32 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1
```

Gambar 3.9 Eksekusi command dbt run

3.5. Query Menggunakan Trino

Setelah ingestion dan dbt run selesai, dilakukan verifikasi melalui Trino CLI untuk memastikan seluruh tahap pipeline berjalan dengan benar. Trino digunakan sebagai query engine karena ia yang menjadi jembatan antara user dan data yang tersimpan di Iceberg dan MinIO.

```
SHOW SCHEMAS FROM iceberg;
SHOW TABLES FROM iceberg.raw;
SHOW TABLES FROM iceberg.mart;
```

Command `SHOW SCHEMAS FROM iceberg` digunakan untuk menampilkan seluruh schema yang terdaftar pada catalog Iceberg. Ekspektasi hasil dari command ini adalah muncul dua schema hasil pipeline yaitu raw sebagai tempat penyimpanan tabel hasil ingestion Spark dan mart sebagai tempat penyimpanan tabel hasil transformasi dbt. Selanjutnya, command `SHOW TABLES FROM iceberg.raw` digunakan untuk memverifikasi bahwa seluruh 8 tabel TPC-H berhasil diingest oleh Spark ke dalam schema raw, yaitu customer, orders, lineitem, nation, part, partsupp, region, dan supplier. Kemudian, command `SHOW TABLES FROM iceberg.mart` digunakan untuk memastikan bahwa tabel Data Mart berhasil dibuat oleh dbt pada schema mart. Munculnya tabel `mart_customer_revenue` menunjukkan bahwa dbt berhasil membaca data dari schema raw, melakukan proses transformasi, dan menuliskan hasilnya kembali ke lokasi yang sesuai.

```

trino> show schemas from iceberg;
Schema
-----
information_schema
mart
raw
system
(4 rows)

Query 20260521_142247_00026_3ppne, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.18 [4 rows, 231B] [22 rows/s, 1.28KiB/s]

trino> show tables from iceberg.raw;
Table
-----
customer
lineitem
nation
orders
part
partsupp
region
supplier
(8 rows)

Query 20260521_142301_00027_3ppne, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.17 [8 rows, 416B] [47 rows/s, 2.42KiB/s]

trino> show tables from iceberg.mart;
Table
-----
mart_customer_revenue
(1 row)

Query 20260521_142328_00028_3ppne, FINISHED, 1 node
Splits: 11 total, 11 done (100.00%)
0.17 [1 rows, 171B] [5 rows/s, 983B/s]

```

Gambar 3.10 Query menggunakan Trino untuk mengecek *schema*

Untuk menguji database berjalan dengan baik, dapat dicoba menjalankan query berikut:

```
SELECT * FROM iceberg.raw.customer LIMIT 5;
```

```

trino> SELECT * FROM iceberg.raw.customer LIMIT 5;
_c0 | _c1 | _c2 | _c3 | _c4 | _c5 | _c6 | _c7
-----
1 | Customer#000000001 | IVhzIApeRb ot,c,E | 15 | 25-989-741-2988 | 711.56 | BUILDING | to the even, regular platelets. regular, ironic epitaph
2 | Customer#000000002 | XSTf4,NcwDVahNe6tEgwwfRchLXak | 13 | 23-768-687-3665 | 121.65 | AUTOMOBILE | l accounts. blithely ironic theodolites integrate boldl
3 | Customer#000000003 | MG9kdTD2MBHn | 1 | 11-719-748-3364 | 7498.12 | AUTOMOBILE | deposits eat slyly ironic, even instructions. express
4 | Customer#000000004 | XxVSJsLAGtn | 4 | 14-128-190-5944 | 2866.83 | MACHINERY | requests. final, regular ideas sleep final accou
5 | Customer#000000005 | KvpYuHCplRB84lgAiGV6sYpZq7Tj | 3 | 13-750-942-6364 | 794.47 | HOUSEHOLD | n accounts will have to unwind. foxes cajole accor
(5 rows)

```

Gambar 3.11 Contoh query menggunakan Trino untuk verifikasi data

4. Lessons Learned

4.1. Perbandingan Storage CSV vs Parquet

Salah satu keuntungan utama dari penggunaan format Parquet dalam arsitektur Data Lakehouse adalah efisiensi penyimpanan dibandingkan format CSV. Pada proyek ini, dataset TPC-H yang di-generate menggunakan DBGEN dengan scale factor 10 menghasilkan 8 file berformat .tbl dengan total ukuran sekitar 11 GB. Setelah seluruh data diingest ke Data Lakehouse menggunakan Apache Spark dan disimpan dalam format Parquet di MinIO, total ukuran keseluruhan bucket

warehouse (mencakup file Parquet raw, Data Mart, dan metadata Iceberg) hanya sebesar 2,4 GB. Ini berarti terjadi pengurangan ukuran sebesar sekitar 78.18% atau sekitar 4.58 kali lebih kecil dibandingkan format CSV aslinya.

```
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/raw_tbl$ ls
customer.tbl  lineitem.tbl  load_lakehouse.py  nation.tbl  orders.tbl  part.tbl  partsupp.tbl  region.tbl  supplier.tbl
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/raw_tbl$ du -lh *.tbl
234M customer.tbl
7.3G lineitem.tbl
4.0K nation.tbl
1.7G orders.tbl
233M part.tbl
1.2G partsupp.tbl
0 region.tbl
14M supplier.tbl
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/raw_tbl$ du -ch *.tbl | grep total$
11G total
```

Gambar 4.1 Ukuran tiap file CSV dan ukuran total seluruh file CSV

```
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/minio_storage/warehouse/raw$ ls
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/minio_storage/warehouse/raw$ du -lh *
6.9M customer/data/00000-13-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/b6868c12-f1c2-443d-befc-belf9c2cae3d
6.9M customer/data/00000-13-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00001-14-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/b4258625-f0eb-4539-a96b-bd7a4efcea28
6.9M customer/data/00001-14-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00002-15-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/4341fa75-2e72-49d5-814e-002fd48577da
6.9M customer/data/00002-15-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.8M customer/data/00003-16-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/c8d17eda-122d-4357-ac48-9a0d8e10f631
6.8M customer/data/00003-16-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00004-17-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/9b96c9f5-ebf1-4574-ad62-45c280ad7b97
6.9M customer/data/00004-17-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00005-18-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/7dfb5b42-257a-490d-aad6-51311470f3f1
6.9M customer/data/00005-18-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00006-19-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/ae3b28bc-570a-42bc-a334-396ff2ac96f6
6.9M customer/data/00006-19-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.9M customer/data/00007-20-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/ddd62960-a168-4c08-aeb7-e8552f89495a
6.9M customer/data/00007-20-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.8M customer/data/00008-21-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/c02d0156-5ad2-4b32-b672-5c8c1e78feb7
6.8M customer/data/00008-21-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.8M customer/data/00009-22-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/f151eb0e-3fa5-4549-9261-8863bac2d02b
6.8M customer/data/00009-22-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
6.8M customer/data/00010-23-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/9268a974-4a46-4ac2-9044-3ce0c65e2af5
6.8M customer/data/00010-23-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
5.4M customer/data/00011-24-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet/7bfff9c8-bb4d-4c50-98eb-fdff05f535da
5.4M customer/data/00011-24-82184f4c-8cf1-4195-8f0d-73eb7a272731-0-00001.parquet
81M customer/data
4.0K customer/metadata/00000-302d61u3-a89d-455b-8144-f0e69a863ccb.metadata.json
12K customer/metadata/070d6fcb-d093-4a03-bc65-fell18c378d38-m0.avro
8.0K customer/metadata/snap-521323718449740725-1-a78d6fcb-d093-4a03-bc65-fell18c378d38.avro
24K customer/metadata
81M customer
29M lineitem/data/00000-84-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet/cf894947-5eef-4d2f-a3a3-87698cbc379e
29M lineitem/data/00000-84-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet
29M lineitem/data/00001-85-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet/9209b1ed-f931-447f-86bd-6498022643f8
29M lineitem/data/00001-85-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet
29M lineitem/data/00002-86-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet/7dc986d9-1734-4415-9eca-223a5d87af0e
29M lineitem/data/00002-86-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet
29M lineitem/data/00003-87-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet/89dbf29b-6db8-4f72-b303-a61baf3c86ba
29M lineitem/data/00003-87-e23a9dab-d47e-47ac-8b74-9d939350c33e-0-00001.parquet
```

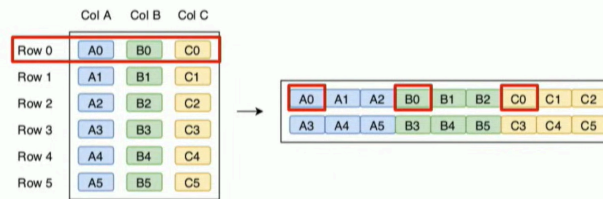
Gambar 4.2 Contoh daftar ukuran file Parquet di Lakehouse

```
(dolt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/minio_storage/warehouse/raw$
du -ch * | grep total$
2.4G total
```

Gambar 4.3 Ukuran total seluruh file Parquet di Lakehouse

Pengurangan ukuran yang drastis ini terjadi karena pendekatan yang dilakukan Parquet dalam menyimpan datanya. Pertama, Parquet menyimpan datanya dalam bentuk *hybrid columnar* (melakukan *horizontal partitioning* pada *row* data terlebih dahulu, misal *row* 1-100, 101-200, dst. lalu menyimpan tiap partisi secara *columnar*). Pendekatan ini memungkinkan Parquet untuk memperoleh *the best of both worlds*: 1) Melakukan query analitik dengan cepat (karena query analitik umumnya hanya *select* beberapa kolom saja), dan 2) Dapat melakukan rekonstruksi *row* data dengan relatif cepat (karena data nilai seluruh atribut dari suatu *row* pasti ada di satu partisi yang sama).

Hybrid



- Horizontal & vertical partitioning
- Used by Parquet & ORC
- Best of both worlds

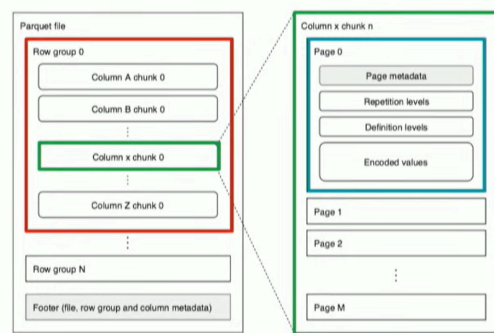
databricks

Gambar 4.4 Pendekatan Parquet dalam menyimpan data ([Sumber](#))

Pada gambar diatas, terdapat 2 partisi columnar, yaitu [A0, A1, A2, B0, B1, B2, C0, C1, C2] dan [A3, A4, A5, B3, B4, B5, C3, C4, C5]. Hasil kumpulan partisi *columnar* ini kemudian disimpan di beberapa file Parquet, dimana satu file Parquet berisi beberapa partisi sekaligus. Sebuah file Parquet memiliki struktur seperti gambar dibawah:

Parquet: data organization

- Data organization
 - Row-groups (default 128MB)
 - Column chunks
 - Pages (default 1MB)
 - Metadata
 - Min
 - Max
 - Count
 - Rep/def levels
 - Encoded values



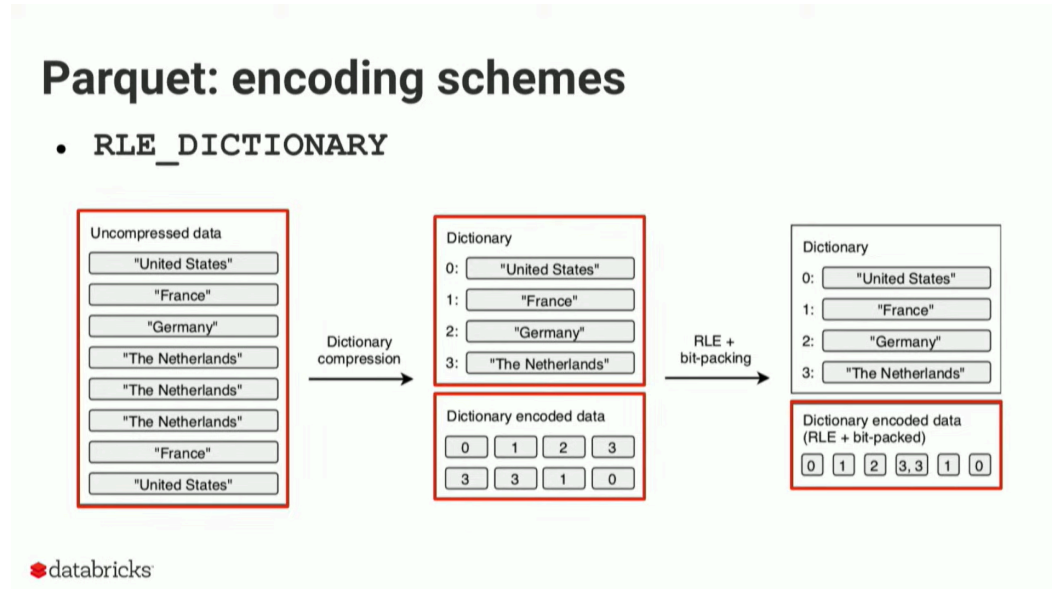
databricks

Gambar 4.5 Struktur file Parquet ([Sumber](#))

Suatu file Parquet terdiri dari kumpulan *row group* (dimana satu *row group* menyimpan satu partisi *columnar*) dan satu Footer yang berisi metadata dari file, *row group*, dan *column* (misalnya nilai *min* dan *max* dari tiap atribut pada seluruh *row group*). Satu *row group* terdiri dari beberapa *column chunk* (dimana satu

chunk menyimpan nilai data satu kolom tabel secara kontinu, misal [A0, A1, A2]), dan satu *column chunk* terdiri dari *page metadata*, *repetition levels*, *definition levels*, dan *encoded values*. Page metadata berisi metadata untuk column chunk tersebut (misalnya *min* dan *max* dari kolom yang disimpan), *repetition levels* dan *definition levels* berisi angka-angka untuk merekonstruksi *nested tuples* (lebih jelasnya dapat dilihat di paper Dremel), dan *encoded values* berisi nilai dari kolom itu sendiri (misal [A0, A1, A2]). Teknik *encoding* pada bagian *encoded values* inilah yang menjadi kunci bagaimana Parquet dapat menyimpan data dengan jauh lebih efisien dibanding data mentah.

Parquet memiliki beberapa skema encoding, yang paling umum diantaranya adalah *Plain* dan *RLE Dictionary*. Pada skema *encoding Plain*, data hanya disimpan dengan cara biasa yaitu secara kontigu (bisa secara *fixed-width* yaitu *back-to-back*, atau secara *non fixed-width* yaitu dengan menambahkan *prefix* di setiap *value* yang menyatakan panjang bit dari *value* tersebut). Sedangkan pada skema *RLE Dictionary*, diterapkan beberapa teknik yaitu *dictionary compression*, *run-length encoding*, dan *bit-packing*. *RLE Dictionary* mengasumsikan terdapat *value* yang diulang pada suatu *column chunk*, sehingga semakin banyak *value* yang diulang (yaitu semakin sedikit *unique value* pada suatu *column chunk*) maka *RLE Dictionary* semakin efektif.



Gambar 4.6 Mekanisme RLE Dictionary ([Sumber](#))

Pada tahap *dictionary compression*, Parquet mencatat *unique values* apa saja yang ada pada *column chunk*, lalu memetakan tiap *unique value* ke suatu angka. Informasi pemetaan *unique value* ke angka ini disimpan di *dictionary*, sehingga semua *value* kemudian dapat digantikan dengan angka berdasarkan *dictionary*

tersebut. Kemudian, diterapkan *run-length encoding* yaitu menggantikan seluruh angka yang diulang secara berturut-turut menjadi pasangan [angka, jumlah/*count*]. Misalnya, pada gambar diatas angka 3 (yang menyatakan “The Netherlands”) diulang tiga kali, sehingga dapat digantikan dengan pasangan [3, 3]. Terakhir, dilakukan bit-packing yaitu menggunakan jumlah bit sesedikit mungkin sesuai yang dibutuhkan. Misalnya, karena *dictionary* hanya berisi 4 *unique value* maka panjang bit yang diperlukan hanya 2 bit saja. Mekanisme RLE Dictionary inilah yang dapat mengubah kumpulan value berukuran besar menjadi ukuran yang jauh lebih kecil.

Kita dapat melihat skema encoding yang digunakan untuk tiap kolom pada suatu file Parquet dengan melakukan query ke metadata Parquet. Kami menggunakan program Python sederhana berikut untuk melakukan query ke salah satu file Parquet:

```
import duckdb
con = duckdb.connect()

parquet_file =
"00000-13-d6035e60-7f09-48d6-b0e2-e8404082ca15-0-00001.parquet/*
/part.1"

result = con.execute(f"""
    SELECT
        path_in_schema AS column_name,
        compression,
        encodings
    FROM parquet_metadata('{parquet_file}')
""").fetchdf()

print(result)
```

Setelah Python *script* diatas dijalankan di direktori *raw* Parquet untuk tabel *customer*, diperoleh hasil sebagai berikut:

```

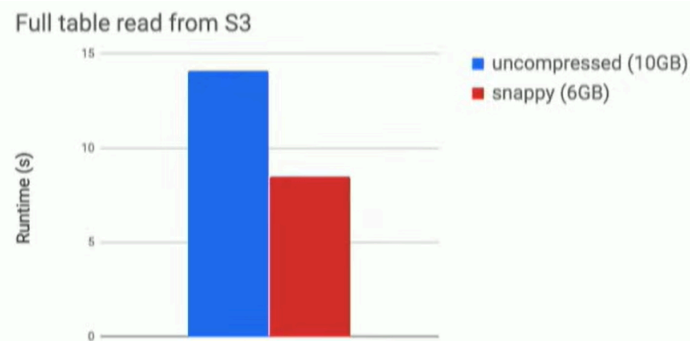
(dbt-env) irfansidiq@LAPTOP-69GN056N: /mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/minio_storage/warehouse/raw/customer/data$ python3
Python 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> import duckdb
>>> con = duckdb.connect()
>>> parquet_file = "00000-13-d6035e60-7f09-48d6-b0e2-e8404082ca15-0-00001.parquet"
>>> result = con.execute(f"""
...     SELECT
...     path_in_schema AS column_name,
...     compression,
...     encodings
... FROM ..
... FROM parquet_metadata('{parquet_file}')
... """).fetchdf()
>>> print(result)
  column_name compression      encodings
0      _c0      ZSTD      BIT_PACKED, RLE, PLAIN
1      _c1      ZSTD      BIT_PACKED, RLE, PLAIN
2      _c2      ZSTD      BIT_PACKED, RLE, PLAIN
3      _c3      ZSTD  PLAIN_DICTIONARY, BIT_PACKED, RLE
4      _c4      ZSTD      BIT_PACKED, RLE, PLAIN
5      _c5      ZSTD      BIT_PACKED, RLE, PLAIN
6      _c6      ZSTD  PLAIN_DICTIONARY, BIT_PACKED, RLE
7      _c7      ZSTD      BIT_PACKED, RLE, PLAIN
8      _c8      ZSTD      BIT_PACKED, RLE, PLAIN
>>> |

```

Gambar 4.7 Metadata *compression* dan *encoding* dari suatu file Parquet

Dapat dilihat bahwa kebanyakan kolom menggunakan encoding [BIT_PACKED, RLE, PLAIN], dan hanya kolom `_c3` dan `_c7` yang menggunakan encoding [PLAIN_DICTIONARY, BIT_PACKED, RLE]. Encoding PLAIN dipilih ketika sebuah kolom memiliki sangat banyak *unique values* (contoh: ID customer, nama customer, revenue, dll.), sedangkan encoding PLAIN_DICTIONARY dipilih ketika sebuah kolom memiliki sedikit *unique values* (banyak *value* yang diulang, contoh: kolom bertipe enum). Pada contoh tabel *customer* diatas, kolom `_c3` adalah kolom Market Segment yang hanya memiliki 5 possible values (AUTOMOBILE, BUILDING, MACHINERY, FURNITURE, HOUSEHOLD), sedangkan kolom `_c7` adalah kolom Nation Key yang hanya memiliki 25 possible values (berupa ID 0-24 yang masing-masing merepresentasikan suatu negara).

Selain skema encoding, pada Parquet juga diterapkan *page compression*. Metode ini memang membuat ukuran file akhir menjadi lebih kecil, namun menambahkan *overhead* untuk *decompression* nantinya. Walaupun begitu, dengan ukuran file yang lebih kecil maka overhead I/O menjadi lebih kecil pula, sehingga dengan kompresi yang tepat maka total waktu pemrosesan data menggunakan page compression tetap bisa lebih singkat.



Gambar 4.8 Perbandingan waktu pemrosesan data awal dengan data yang telah dikompres menggunakan Snappy ([Sumber](#))

Kita dapat melihat informasi tambahan dari *compression* suatu kolom menggunakan *parquet-tools* dengan contoh *command* sebagai berikut:

```
parquet-tools inspect
00000-13-d6035e60-7f09-48d6-b0e2-e8404082ca15-0-00001.parquet/*/
part.1
```

Setelah menjalankan *command* diatas, muncul tampilan sebagai berikut:

```
(dbt-env) irfansidiq@APTOP-69CH856M:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/data/minio_storage/warehouse/raw/customer/data$ pa
rquet-tools inspect 00000-13-d6035e60-7f09-48d6-b0e2-e8404082ca15-0-00001.parquet/*/part.1

##### file meta data #####
created_by: parquet-mr version 1.15.0 (build 4665401d36e468c988322f78621da7c2d1c22ede)
num_columns: 9
num_rows: 127973
num_row_groups: 1
format_version: 1.0
serialized_size: 1983

##### Columns #####
_c0
_c1
_c2
_c3
_c4
_c5
_c6
_c7
_c8

##### Column(_c0) #####
name: _c0
path: _c0
max_definition_level: 1
max_repetition_level: 0
physical_type: INT32
logical_type: None
converted_type (legacy): NONE
compression: ZSTD (space_saved: 37%)

##### Column(_c1) #####
name: _c1
path: _c1
max_definition_level: 1
max_repetition_level: 0
physical_type: BYTE_ARRAY
logical_type: String
converted_type (legacy): UTF8
```

Gambar 4.9 Metadata *compression* dan *encoding* dari suatu file Parquet

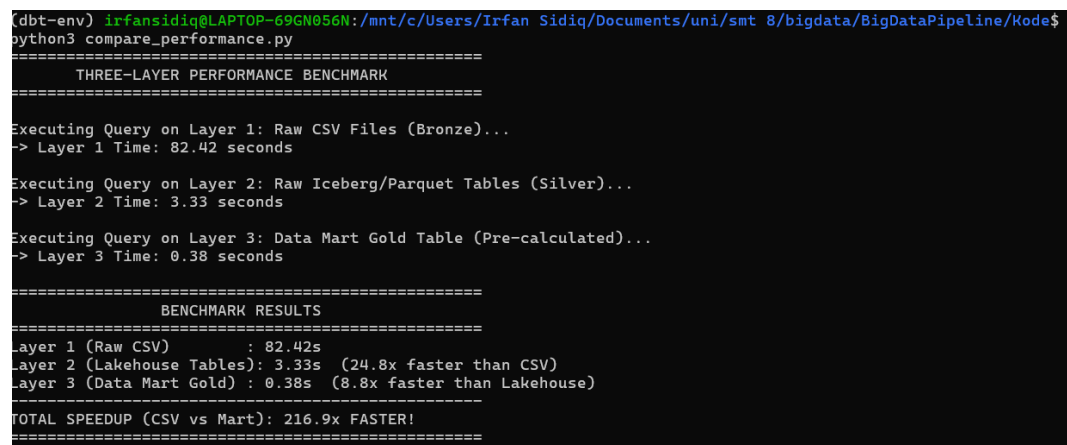
Dapat dilihat bahwa mekanisme *compression* yang dilakukan pada seluruh kolom adalah ZSTD, yaitu mekanisme compression yang memperoleh the best of both world: rasio kompresi yang tinggi (setara GZIP) dan kecepatan dekompresi yang tinggi (setara Snappy). Pada contoh di atas, kolom *_c1* merupakan kolom nama *customer* yang berpola *Customer#000000001*, *Customer#000000002*, dst. sehingga walaupun tidak dapat diterapkan *encoding* RLE Dictionary (karena semua *value* unik), ZSTD tetap dapat melakukan kompresi yang sangat bagus dengan cara merepresentasikan pola yang sama (yaitu *Customer#00000*) dengan bit yang sangat singkat, sedangkan pola yang berbeda (misalnya angka dibelakang *Customer#00000*) direpresentasikan dengan bit yang relatif lebih panjang.

4.2. Perbandingan Kecepatan Query Data CSV vs Data Lakehouse vs Data Mart

Untuk mengukur kecepatan eksekusi query terhadap data CSV mentah, data lakehouse, dan data mart, dijalankan *python script* `compare_performance.py` yang ada di folder `Kode`. Script tersebut nantinya akan menjalankan query untuk menghitung total order dan total revenue dari tiap customer untuk 30.000 customer pertama. Query tersebut ditulis sebagai berikut:

```
SELECT
  c._c0 AS customer_key,
  c._c1 AS customer_name,
  count(distinct o._c0) AS total_orders,
  sum(l._c5 * (1 - l._c6)) AS total_revenue
FROM
  customer c
  JOIN orders o ON c._c0 = o._c1
  JOIN lineitem l ON o._c0 = l._c0
WHERE c._c0 < 30000
GROUP BY 1, 2
```

Query ini akan dijalankan menggunakan DuckDB yang membuat koneksi ke MinIO secara langsung tanpa melalui Trino (alasan mengapa kami perlu menggunakan DuckDB dibanding Trino untuk mengetes *performance* akan dijelaskan di bagian *challenges*). Ketika python script ini dijalankan di terminal, maka akan menghasilkan output sebagai berikut:



```
(dbt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/Kode$
python3 compare_performance.py
=====
THREE-LAYER PERFORMANCE BENCHMARK
=====
Executing Query on Layer 1: Raw CSV Files (Bronze)...
-> Layer 1 Time: 82.42 seconds

Executing Query on Layer 2: Raw Iceberg/Parquet Tables (Silver)...
-> Layer 2 Time: 3.33 seconds

Executing Query on Layer 3: Data Mart Gold Table (Pre-calculated)...
-> Layer 3 Time: 0.38 seconds

=====
BENCHMARK RESULTS
=====
Layer 1 (Raw CSV) : 82.42s
Layer 2 (Lakehouse Tables): 3.33s (24.8x faster than CSV)
Layer 3 (Data Mart Gold) : 0.38s (8.8x faster than Lakehouse)
=====
TOTAL SPEEDUP (CSV vs Mart): 216.9x FASTER!
=====
```

Gambar 4.10 Hasil benchmark query untuk raw CSV vs Lakehouse vs Mart

Berdasarkan tangkapan layar di atas, ditemukan bahwa query yang dilakukan dengan membaca file raw CSV secara langsung di MinIO membutuhkan waktu 82,42 detik, sedangkan query yang dilakukan dengan membaca file raw Parquet di Lakehouse MinIO membutuhkan waktu 3,33 detik (24,8 kali lipat lebih cepat

dari CSV). Selanjutnya, query yang sama ketika dilakukan dengan membaca file Parquet di Mart MinIO membutuhkan waktu 0,38 detik saja (8,8 kali lipat lebih cepat dari Lakehouse, dan 216,9 kali lipat lebih cepat dari CSV).

Perbedaan waktu eksekusi yang sangat signifikan ini disebabkan oleh beberapa hal. Pertama, karena Parquet mereduksi ukuran file menjadi jauh lebih kecil, overhead I/O juga menjadi jauh lebih kecil, yang menyebabkan waktu eksekusi query lebih cepat. Kedua, seperti yang telah dijelaskan pada perbandingan storage diatas, Parquet menyimpan data secara *hybrid columnar* dan menyimpan metadata dari keseluruhan file (*footer*) maupun metadata tiap *column chunk* yang berisi misalnya *min* dan *max* dari value yang terdapat pada chunk tersebut. Dengan menyimpan data secara *hybrid columnar* maka query analitik dapat dilakukan dengan jauh lebih cepat (karena query analitik umumnya hanya men-*select* beberapa kolom saja sehingga disk dapat mengambil data secara kontigu dengan memanfaatkan *caching*, *pre-fetch*, dll.), dan dengan menyimpan metadata yang berisi informasi *min* dan *max* maka dapat dilakukan *predicate pushdown* sehingga basis data hanya membaca suatu file atau suatu *column chunk* bila *value* yang dicari berada di dalam interval *min* dan *max* yang tertera pada metadata.

Selain itu, Parquet juga menerapkan *dictionary filtering* yaitu dengan melihat apakah *value* yang dicari ada pada dictionary suatu *file* atau *column chunk*. Dictionary hanya berisi daftar unique value yang ada pada satu file (di *footer*) atau *column chunk* tersebut saja, sehingga bila *value* yang dicari tidak ada di dictionary maka basis data tidak perlu membaca file atau *column chunk* tersebut. Pada Parquet kita juga dapat melakukan partitioning berdasarkan suatu atribut (misalnya tanggal), sehingga bila kita sudah tahu pasti query analitik apa yang diperlukan maka kita dapat menggunakan partitioning untuk menstrukturkan file direktori berdasarkan atribut yang diperlukan untuk filtering. Kekurangannya yaitu bila suatu saat query analitiknya berubah, maka struktur direktori berdasarkan atribut justru menjadi penghambat karena atribut direktori tidak sesuai dengan atribut yang digunakan untuk filtering pada query, sehingga query menjadi jauh lebih lambat.

Optimization: partitioning

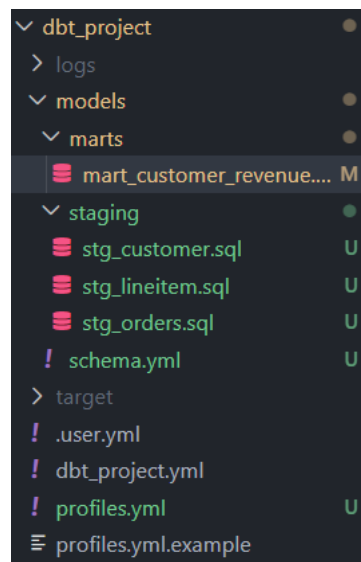
- Embed predicates in directory structure

```
df.write.partitionBy("date").parquet(...)  
  
./example_parquet_file/date=2019-10-15/...  
./example_parquet_file/date=2019-10-16/...  
./example_parquet_file/date=2019-10-17/part-00000-...-475b15e2874d.c000.snappy.parquet
```

Gambar 4.11 Contoh partitioning berdasarkan suatu atribut

4.3. Memanfaatkan Fitur-Fitur dbt

dbt menyediakan berbagai fitur yang mendukung pengembangan data pipeline dengan pendekatan *software engineering* modern sehingga *pipeline* menjadi lebih terstruktur, mudah dipelihara, dan reusable. Salah satu fitur utamanya adalah *models*, yaitu transformasi data yang ditulis dalam file SQL terpisah sehingga setiap proses transformasi dapat dibagi menjadi modul-modul kecil yang lebih mudah dibaca dan dikelola.



Gambar 4.12 Contoh struktur folder *models* di dbt

(Catatan: Folder *staging* diatas hanya digunakan sebagai demonstrasi, dan tidak akan di-*push* ke Github)

dbt juga menyediakan *dependency management* melalui fungsi `ref()` yang memungkinkan hubungan antar model dibangun secara otomatis tanpa perlu meng-*hardcode* nama tabel secara manual. Dengan begitu, ketika nama tabel

berubah kita hanya perlu mengubah `ref()` di config saja tanpa perlu mengubah file-file lain, sehingga kode mudah dipelihara. Untuk proyek ini, karena skema kami sangat sederhana (hanya tabel-tabel *raw data* dan satu tabel *mart*) kami belum perlu menggunakan fitur `ref()` tersebut. Namun sebagai contoh, bila misalnya ditambahkan tabel-tabel antara untuk menjembatani tabel *raw data* dengan tabel *mart*:

```
models/staging/stg_customer.sql
```

```
{{ config(materialized='view') }}

SELECT
    _c0 AS customer_key,
    _c1 AS customer_name,
    _c3 AS market_segment
FROM {{ source('raw', 'customer') }}
```

```
models/staging/stg_orders.sql
```

```
{{ config(materialized='view') }}

SELECT
    _c0 AS order_key,
    _c1 AS customer_key
FROM {{ source('raw', 'orders') }}
```

```
models/staging/stg_lineitem.sql
```

```
{{ config(materialized='view') }}

SELECT
    _c0 AS order_key,
    _c4 AS quantity,
    _c5 AS extended_price,
    _c6 AS discount
FROM {{ source('raw', 'lineitem') }}
```

Dengan mendefinisikan tiga file diatas, kita dapat menggunakan `ref()` untuk *me-refer* tabel-tabel di atas untuk membuat tabel *mart*:

```
models/marts/mart_customer_revenue.sql
```

```
{{ config(materialized='table') }}
```

```

WITH customers AS (
    SELECT * FROM {{ ref('stg_customer') }}
),
orders AS (
    SELECT * FROM {{ ref('stg_orders') }}
),
lineitems AS (
    SELECT * FROM {{ ref('stg_lineitem') }}
)

SELECT
    c.customer_key,
    c.customer_name,
    c.market_segment,
    COUNT(DISTINCT o.order_key) AS total_orders,
    SUM(l.extended_price) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_key = o.customer_key
JOIN lineitems l ON o.order_key = l.order_key
GROUP BY
    c.customer_key,
    c.customer_name,
    c.market_segment
ORDER BY
    total_revenue DESC

```

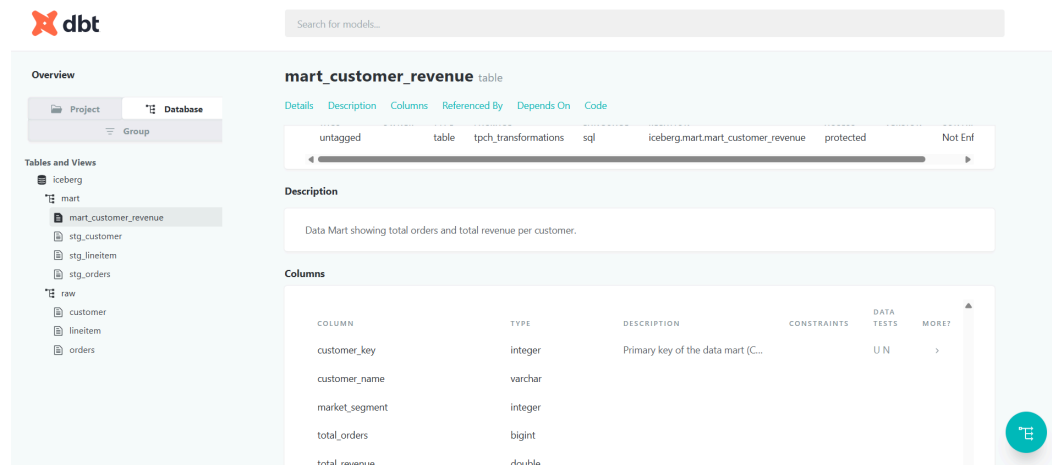
Selain itu, kita juga dapat menjalankan command berikut untuk men-*generate* dokumentasi dari dbt kita:

```
dbt docs generate --profiles-dir .
```

Hasilnya dapat diakses menggunakan *command* berikut untuk membuka UI dokumentasi dbt di localhost:8000:

```
dbt docs serve --profiles-dir . --port 8000
```

Melalui UI dbt, kita dapat melihat skema dari basis data kita. Kita juga dapat melihat lineage graph dari tabel-tabel yang telah kita buat menggunakan fitur `ref()` tadi:



Gambar 4.13 Tampilan UI Dokumentasi dbt



Gambar 4.14 Tampilan *lineage graph* (DAG) model di dbt

dbt juga mendukung penggunaan Jinja templating dan *macros* untuk membuat logic SQL yang dinamis dan dapat di-*reuse* pada berbagai model. Pada proyek ini, kami belum memanfaatkan fitur macros tersebut karena logika SQL pada query yang kami gunakan masih sederhana dan tidak diulang. Namun, kedepannya bila misalnya ditambahkan *macro* berikut:

```
macros/calculate_discounted_price.sql

{% macro calculate_discounted_price(extended_price, discount) %}
    ({{ extended_price }} * (1 - {{ discount }}))
{% endmacro %}
```

Maka kita dapat menggunakannya untuk melakukan query seperti:

```
models/marts/mart_customer_revenue.sql
```

```
{{ config(materialized='table') }}

WITH customers AS (
    SELECT * FROM {{ ref('stg_customer') }}
),

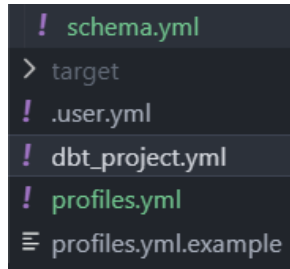
orders AS (
    SELECT * FROM {{ ref('stg_orders') }}
),

lineitems AS (
    SELECT * FROM {{ ref('stg_lineitem') }}
)

SELECT
    c.customer_key,
    c.customer_name,
    c.market_segment,
    COUNT(DISTINCT o.order_key) AS total_orders,
    SUM({{ calculate_discounted_price('l.extended_price',
    'l.discount') }}) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_key = o.customer_key
JOIN lineitems l ON o.order_key = l.order_key
GROUP BY
    c.customer_key,
    c.customer_name,
    c.market_segment
ORDER BY
    total_revenue DESC
```

Dengan begitu, kode menjadi jauh lebih mudah untuk di-*reuse* dan dipelihara (tinggal mengganti definisi *macros* untuk menerapkan perubahannya ke semua query), dan juga lebih mudah dibaca dibanding query yang kompleks tanpa menggunakan *macros*.

dbt juga menggunakan file-file konfigurasi seperti `schema.yml`, `profiles.yml`, `dbt_project.yml`, dan sebagainya sehingga kita dapat dengan mudah mengganti kode yang terdapat di file-file tersebut untuk menerapkan perubahan konfigurasi tanpa perlu terjun langsung menjalankan command di CLI tiap *container worker* Trino (mirip dengan Kubernetes).



Gambar 4.15 Contoh file-file konfigurasi di dbt

Fitur *package management* juga memungkinkan penggunaan *package* atau library dari komunitas dbt (mirip dengan *pip* di Python atau *npm* di Node.js), sehingga kita dapat menggunakan langsung fungsi-fungsi *library* tersebut di kode kita. Dalam proyek ini kami belum memerlukan library tambahan, namun misalnya saja bila kedepannya kita perlu menambahkan ID unik sebagai primary key untuk tabel `mart_customer_revenue` berdasarkan atribut `customer_key` dan `market_segment`, maka kita bisa menambahkan file `packages.yml` di *root directory* yang berisi *library* yang kita inginkan:

```
packages.yml

packages:
  - package: dbt-labs/dbt_utils
    version: 1.1.1
```

Lalu kita dapat menginstal package tersebut dengan command:

```
dbt deps
```

Package tersebut kemudian bisa kita gunakan di kode query dbt kita:

```
models/marts/mart_customer_revenue.sql

{{ config(materialized='table') }}

...

SELECT
  {{ dbt_utils.generate_surrogate_key(['c.customer_key',
  'c.market_segment']) }} AS customer_sk,
  c.customer_key,
  c.customer_name,
  c.market_segment,
  ...
```


Untuk memastikan kualitas data tetap terjaga selama pipeline berjalan, digunakan pula mekanisme data testing seperti *unique* dan *not_null* sehingga tidak ada data yang duplikat maupun data dengan banyak *missing values*. Menjaga kualitas data sangat penting agar *pipeline* tidak gagal di tengah jalan karena data yang kotor. Walaupun *pipeline*-nya ternyata berhasil berjalan dengan data yang kotor, maka jadinya database lakehouse atau data mart kita terisi data yang kotor, yang tentunya sangat tidak kita inginkan karena menghambat pekerjaan analitik atau bahkan memberikan hasil yang salah ketika di-*query*. Dalam proyek ini, kami menggunakan tes *unique* dan *not_null* yang bisa dijalankan dengan command:

```
dbt test
```

```
(dbt-env) irfansidiq@LAPTOP-69GN056N:/mnt/c/Users/Irfan Sidiq/Documents/uni/smt 8/bigdata/BigDataPipeline/Kode/dbt_project$ dbt test
00:39:07 Running with dbt=1.11.10
INFO:trino.auth:keyring module not found. OAuth2 token will not be stored in keyring.
INFO:trino.auth:keyring module not found. OAuth2 token will not be stored in keyring.
00:39:07 Registered adapter: trino=1.10.1
00:39:17 Found 1 model, 2 data tests, 3 sources, 487 macros
00:39:17 Concurrency: 4 threads (target='dev')
00:39:17
00:39:21 [WARNING]: SSL certificate validation is disabled by default. It is legacy behavior which will be changed in future releases. It
is strongly advised to enable 'require_certificate_validation' flag or explicitly set 'cert' configuration to 'True' for security reasons.
You may receive an error after that if your SSL setup is incorrect.
You may opt into the new behavior sooner by setting 'flags.require_certificate_validation' to 'True' in 'dbt_project.yml'.
Visit https://docs.getdbt.com/reference/global-configs/behavior-changes for more information.
00:39:22 1 of 2 START test not_null_mart_customer_revenue_customer_key ..... [RUN]
00:39:22 2 of 2 START test unique_mart_customer_revenue_customer_key ..... [RUN]
00:39:22 1 of 2 PASS not_null_mart_customer_revenue_customer_key ..... [PASS in 0.72s]
00:39:23 2 of 2 PASS unique_mart_customer_revenue_customer_key ..... [PASS in 1.39s]
00:39:23
00:39:23 Finished running 2 data tests in 0 hours 0 minutes and 5.86 seconds (5.86s).
00:39:23 Completed successfully
00:39:23
00:39:23 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
```

Gambar 4.16 Hasil tes *unique* dan *not_null* menggunakan *dbt test*

4.4. Challenges

Salah satu *challenge* terbesar yang kami hadapi saat mengerjakan tugas ini adalah terkait *resource* yang dibutuhkan. Untuk menjalankan query pada data yang berukuran besar (11 GB), seringkali kami mendapatkan pesan error *insufficient memory* karena kapasitas RAM dari laptop kami (yaitu 8 GB untuk alokasi *resource* Docker) tidak cukup untuk menyimpan data sementara selama proses berjalannya query. Query pertama yang kami coba adalah sebagai berikut:

```
SELECT
    c.c_custkey AS customer_key,
    c.c_name AS customer_name,
    COUNT(DISTINCT o.o_orderkey) AS total_orders,
    SUM(l.l_extendedprice * (1 - l.l_discount)) AS total_revenue
FROM iceberg.tpch.customer c
JOIN iceberg.tpch.orders o ON c.c_custkey = o.o_custkey
JOIN iceberg.tpch.lineitem l ON o.o_orderkey = l.l_orderkey
GROUP BY
    c.c_custkey,
```

```
c.c_name ;
```

Pada query diatas, tidak dilakukan sama sekali filtering sehingga data hasil query menjadi sangat besar setelah dilakukan proses join, dan menghasilkan pesan error *insufficient memory*.

Selanjutnya, kami mencoba menambahkan predikat filter `WHERE` agar basis data melakukan *predicate pushdown* terlebih dahulu, sehingga data yang menetap di RAM selama proses query hanya merupakan data yang lolos predikat filter. Query yang dicoba yaitu sebagai berikut:

```
SELECT
    c.c_custkey AS customer_key,
    c.c_name AS customer_name,
    COUNT(DISTINCT o.o_orderkey) AS total_orders,
    SUM(l.l_extendedprice * (1 - l.l_discount)) AS total_revenue
FROM iceberg.tpch.customer c
JOIN iceberg.tpch.orders o ON c.c_custkey = o.o_custkey
JOIN iceberg.tpch.lineitem l ON o.o_orderkey = l.l_orderkey
WHERE c.c_mktsegment = 'AUTOMOBILE'
    AND o.o_orderdate >= DATE '1998-01-01'
GROUP BY
    c.c_custkey,
    c.c_name;
```

Pada query diatas, dilakukan filtering sehingga data yang dipilih hanya data yang memiliki segmen pasar *automobile* dan tanggal pemesanannya lebih dari sama dengan 1 Januari 1998. Namun sayangnya, setelah ditambahkan filter diatas pun hasil datanya masih terlalu banyak sehingga kembali muncul pesan error *insufficient memory*.

Query selanjutnya yang kami cobakan yaitu sebagai berikut:

```
SELECT
    c.c_custkey AS customer_key,
    c.c_name AS customer_name,
    COUNT(DISTINCT o.o_orderkey) AS total_orders,
    SUM(l.order_revenue) AS total_revenue
FROM (
    SELECT c_custkey, c_name
    FROM iceberg.tpch.customer
    WHERE c_mktsegment = 'AUTOMOBILE'
) c
JOIN (
```

```

SELECT o_orderkey, o_custkey
FROM iceberg.tpch.orders
WHERE o_orderdate >= DATE '1998-03-01'
      AND o_orderdate < DATE '1998-04-01'
) o ON c.c_custkey = o.o_custkey
JOIN (
  SELECT
    l_orderkey,
    SUM(l_extendedprice * (1 - l_discount)) AS order_revenue
  FROM iceberg.tpch.lineitem
  GROUP BY l_orderkey
) l ON o.o_orderkey = l.l_orderkey
GROUP BY
  c.c_custkey,
  c.c_name;

```

Pada query diatas, dilakukan *subquery* terlebih dahulu pada masing-masing tabel untuk melakukan filter dan agregat nilai sebelum dilakukan *join*, dan filter pada tabel *orders* diubah menjadi data dengan tanggal pemesanan antara 1 Maret 1998 hingga 1 April 1998 saja. Mengingat bahwa *query optimizer* yang dimiliki basis data sudah canggih, di query sebelumnya pun kemungkinan besar sudah dilakukan *predicate pushdown* dan *aggregate pushdown* oleh *query optimizer*, sehingga query sebelumnya mungkin sebenarnya sama efisiennya dengan *subquery* yang dituliskan dengan eksplisit seperti di atas. Namun karena filter pada tabel *orders* sekarang jauh lebih ketat (yang sebelumnya lebih dari sama dengan 1 Januari 1998 sekarang menjadi antara 1 Maret 1998 hingga 1 April 1998), data hasil query menjadi jauh lebih sedikit sehingga akhirnya query di atas berjalan dengan lancar tanpa menghasilkan pesan error.

Namun untuk pengujian performa query, kami memutuskan untuk menggunakan query yang lebih simpel yaitu:

```

SELECT
  c._c0 AS customer_key,
  c._c1 AS customer_name,
  count(distinct o._c0) AS total_orders,
  sum(l._c5 * (1 - l._c6)) AS total_revenue
FROM
  customer c
  JOIN orders o ON c._c0 = o._c1
  JOIN lineitem l ON o._c0 = l._c0
WHERE c._c0 < 30000
GROUP BY 1, 2

```

Selain itu, kami juga memutuskan untuk menggunakan DuckDB untuk keperluan membandingkan performa query, dengan alasan utama yaitu hasil-hasil query sebelumnya sering menyebabkan error *insufficient memory* karena Trino berjalan diatas JVM (Java Virtual Machine), yang ketika dijalankan menggunakan Docker memiliki batas atas penggunaan memori (RAM) yang *strict*. Batas atas penggunaan memori ini memang bisa diganti (misalnya pada file `config.properties` di folder `trino_config`, batas maksimum RAM sudah diganti menjadi 5 GB), namun itupun masih tetap berisiko menghasilkan pesan error karena data CSV mentah berukuran jauh lebih besar yaitu 11 GB. Sedangkan pada DuckDB, query dijalankan menggunakan mekanisme yang mirip seperti *memory swapping* di OS. Ketika RAM yang digunakan tidak cukup, DuckDB tidak akan *crash*, melainkan akan secara otomatis memindahkan sebagian data *intermediate* ke *storage* terlebih dahulu sehingga sisa ruang RAM dapat digunakan untuk melanjutkan proses komputasi. Karena mekanisme inilah, kami tidak pernah mengalami error *insufficient memory* ketika mencoba menggunakan DuckDB.

Kami juga menemukan bahwa Trino didesain untuk mengkoordinasikan query antar ratusan mesin sekaligus, sehingga pada proses eksekusi query terdapat *overhead* dari arsitektur *coordinator* dan *worker* yang dimiliki Trino. Trino juga berjalan di atas JVM, sehingga terdapat overhead lain seperti *garbage collection*, *memory footprint*, dan *JVM startup time*. Di sisi lain, DuckDB didesain untuk berjalan secara lokal sehingga tidak ada overhead *coordinator* maupun *network*. DuckDB juga diimplementasikan menggunakan C++ dengan *Vectorized Execution* (memproses data secara “batch” melalui instruksi *assembly* vektor), sehingga DuckDB dapat memanfaatkan secara maksimal ukuran cache L1/L2 yang dimiliki mesin lokal. Hal ini menyebabkan eksekusi query menggunakan DuckDB lebih cepat dibanding Trino, namun memang tidak mencerminkan penggunaan di industri karena *big data* umumnya pasti memerlukan banyak mesin sekaligus.

Masalah minor lain yang kami temui yaitu karena *container* Iceberg secara *default* menyimpan metadata-nya secara *in-memory*, ketika kami menjalankan `docker compose down` metadata tersebut hilang (walaupun file aslinya masih tetap ada di MinIO) sehingga kami perlu menjalankan ulang *script ingestion* dan *dbt* untuk membuat kembali metadata di Iceberg. Namun masalah ini dapat diselesaikan dengan mudah, yaitu dengan menambahkan properti `CATALOG_URI` di `docker-compose.yml` agar data dituliskan ke lokasi tersebut, serta menambahkan volume Docker untuk Iceberg sehingga lokasi `CATALOG_URI` di-mount ke penyimpanan lokal yang *persistent*.

5. Penutup

5.1. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan dalam tugas proyek teknologi big data ini, dapat diambil beberapa kesimpulan sebagai berikut:

- Arsitektur Data Lakehouse secara *end-to-end* telah berhasil diimplementasikan menggunakan MinIO (sebagai *S3-compatible object storage*), Apache Iceberg (sebagai table format), Apache Spark (untuk data ingestion), Trino (sebagai SQL query engine), dan dbt (sebagai alat transformasi dan orkestrasi data).
- Konversi format penyimpanan dari *row-based* CSV (.tbl) ke *column-based* Parquet (*columnar storage*) di dalam Apache Iceberg terbukti memberikan efisiensi penyimpanan yang sangat signifikan, dengan reduksi ukuran file sebesar 78,18% (dari sekitar 11 GB menjadi hanya 2,4 GB).
- Pengujian performa *query* menggunakan DuckDB membuktikan bahwa Parquet pada Lakehouse mampu mengeksekusi *query* 24,8 kali lipat lebih cepat (3,33 detik) dibandingkan file CSV mentah (82,42 detik). Selanjutnya, *query* yang dilakukan melalui tabel Data Mart (*mart_customer_revenue*) memberikan optimasi performa yang jauh lebih cepat lagi dengan kecepatan *query* 216,9 kali lebih cepat (0,38 detik) dibanding file CSV mentah karena tidak perlu melakukan komputasi *join* sama sekali.
- Penerapan fitur-fitur dbt seperti *ref()*, UI dokumentasi dbt dan *lineage graph*, macros, file konfigurasi dbt, serta otomatisasi pengujian kualitas data (*unique* dan *not_null*) dapat menciptakan pipeline data yang lebih *readable*, *maintainable*, dan *reusable* sesuai dengan standar software engineering modern..

5.2. Saran

Beberapa saran yang dapat diterapkan untuk pengembangan proyek lebih lanjut adalah sebagai berikut:

- Proyek ini masih dijalankan secara lokal saja di *device* masing-masing anggota, sehingga kurang mencerminkan environment *big data* yang sesungguhnya yaitu menyimpan dan memproses data di beberapa mesin sekaligus. Resource yang dapat digunakan (seperti RAM dan CPU) juga akhirnya sangat terbatas, dan tidak memungkinkan untuk pemrosesan data yang lebih berat. Oleh karena itu, kedepannya dapat dilakukan *deployment* ke infrastruktur *cloud* untuk dapat mensimulasikan lingkungan *big data* yang lebih realistis.

- Selain memperbanyak jumlah mesin untuk kekuatan *resource* yang lebih besar dan pemrosesan data yang lebih berat, kedepannya dapat juga dikonfigurasi mekanisme replikasi data untuk *failover* dan *disaster recovery* ketika salah satu node mengalami kegagalan.

6. Referensi

- “MinIO Explained: What It Is, Why It Matters, and How to Use It”. <https://medium.com/@rvnitesh/minio-explained-what-it-is-why-it-matters-and-how-to-use-it-007349b41dce> (Diakses 20 Mei 2026)
- “Kenali Apache Spark”. <https://dqlab.id/kenali-apache-spark-teknologi-hebat-data-engineer> (Diakses 20 Mei 2026)
- “Apa itu Apache Iceberg?”. <https://aws.amazon.com/id/what-is/apache-iceberg/> (Diakses 20 Mei 2026)
- “Trino”. <https://docs.datahub.com/docs/generated/ingestion/sources/trino> (Diakses 20 Mei 2026)
- “Parquet vs CSV: Which Format Should You Choose?”. <https://last9.io/blog/parquet-vs-csv/> (Diakses 21 Mei 2026)
- “What, exactly, is dbt?”. <https://www.getdbt.com/blog/what-exactly-is-dbt> (Diakses 21 Mei 2026)
- “Memory Management in DuckDB”. <https://duckdb.org/2024/07/09/memory-management> (Diakses 23 Mei 2026)
- “The Parquet Format and Performance Optimization Opportunities Boudewijn Braams (Databricks)”. https://www.youtube.com/watch?v=1j8SdS7s_NY (Diakses 24 Mei 2026)